

R-Code Beispiele aus WDDA Vorlesung 6 (Kapitel 6)

WDDA – Datenmanagement und Datenanalyse

Prof. Dr. Ulrich Matter

2026-05-06

Inhaltsverzeichnis

1	Einleitung	1
2	Vorbereitung	2
3	Untersuchung der erklärenden Variablen	2
4	Multiple lineare Regression (TV und Radio)	4
5	Vorhersage mit dem Modell	7
6	Regressionsfläche (Plane of Best Fit)	8
7	Modellgüte und Erklärungskraft	9
8	Residuenanalyse	10
9	Erweiterung: Multiple Regression mit drei Prädiktoren	15
10	Nichtlineare Modelle: Quadratische und polynomiale Regression	17
11	Modellvergleich: R^2 , Adjustiertes R^2 und RSE	19
12	Zusammenfassung	22

1 Einleitung

Dieses Dokument fasst alle R-Code-Beispiele der Folien aus der WDDA FS2026 Vorlesung 6 (Kapitel 6) zusammen. Der Fokus liegt auf der multiplen linearen Regression, einer Erweiterung der einfachen linearen Regression auf mehrere Prädiktoren.

Die multiple lineare Regression ist ein statistisches Verfahren, das den Zusammenhang zwischen einer abhängigen Variable und mehreren unabhängigen Variablen modelliert. Im Gegensatz zur einfachen linearen Regression, die nur einen Prädiktor verwendet, können mit der multiplen Regression komplexere Zusammenhänge untersucht werden.

Zentrale Konzepte in diesem Bereich sind:

1. **Multiple Regression mit mehreren Prädiktoren:** Wie modelliert man den Einfluss mehrerer Variablen?
2. **Residuenanalyse und Modellgüte:** Wie gut passt das Modell zu den Daten?
3. **Prognose und Fehlerbereich:** Wie genau sind die Vorhersagen des Modells?

4. **Nichtlineare und polynomiale Regression:** Wie modelliert man nichtlineare Zusammenhänge?
5. **Modellvergleich:** Wie vergleicht man verschiedene Regressionsmodelle?

2 Vorbereitung

```
# Pakete laden
library(scatterplot3d)
library(rgl)
library(car)
library(readxl)

# Daten laden
data <- read_excel("data/WDDA_06.xlsx", sheet = "Advertising")

# variablen direkt anwählbar machen
attach(data)

## tibble [200 x 4] (S3: tbl_df/tbl/data.frame)
## $ TV      : num [1:200] 230.1 44.5 17.2 151.5 180.8 ...
## $ radio   : num [1:200] 37.8 39.3 45.9 41.3 10.8 48.9 32.8 19.6 2.1 2.6 ...
## $ newspaper: num [1:200] 69.2 45.1 69.3 58.5 58.4 75 23.5 11.6 1 21.2 ...
## $ sales   : num [1:200] 22.1 10.4 9.3 18.5 12.9 7.2 11.8 13.2 4.8 10.6 ...
```

R-Erklärung:

- Die Funktion `library()` lädt ein installiertes Paket in die aktuelle R-Sitzung.
- Das Paket `scatterplot3d` ermöglicht die Erstellung von 3D-Streudiagrammen.
- Das Paket `rgl` bietet interaktive 3D-Visualisierungen.
- Das Paket `car` (Companion to Applied Regression) enthält Funktionen für die Regressionsanalyse.
- Mit `read_excel()` aus dem Paket `readxl` können Excel-Dateien eingelesen werden.
- Die Funktion `attach()` macht die Variablen eines Dataframes direkt ansprechbar, ohne dass man den Dataframe-Namen voranstellen muss.
- Die Funktion `str()` zeigt die Struktur eines Objekts, einschliesslich Datentypen und Dimensionen.

Hinweis zu Datenpfaden: In R-Markdown-Dateien werden relative Pfade vom Speicherort der Datei aus interpretiert, daher der Pfad `../data/WDDA_06.xlsx`. In regulären R-Skripten würde der Pfad relativ zum Arbeitsverzeichnis angegeben werden.

3 Untersuchung der erklärenden Variablen

Bevor wir mit der multiplen Regression beginnen, untersuchen wir die Beziehungen zwischen den Variablen.

```
# Korrelationen
cor_tv <- cor(sales, TV)
cor_radio <- cor(sales, radio)
cat("Korrelation (TV vs. Sales):", round(cor_tv, 4), "\n")

## Korrelation (TV vs. Sales): 0.7822

cat("Korrelation (Radio vs. Sales):", round(cor_radio, 4), "\n\n")

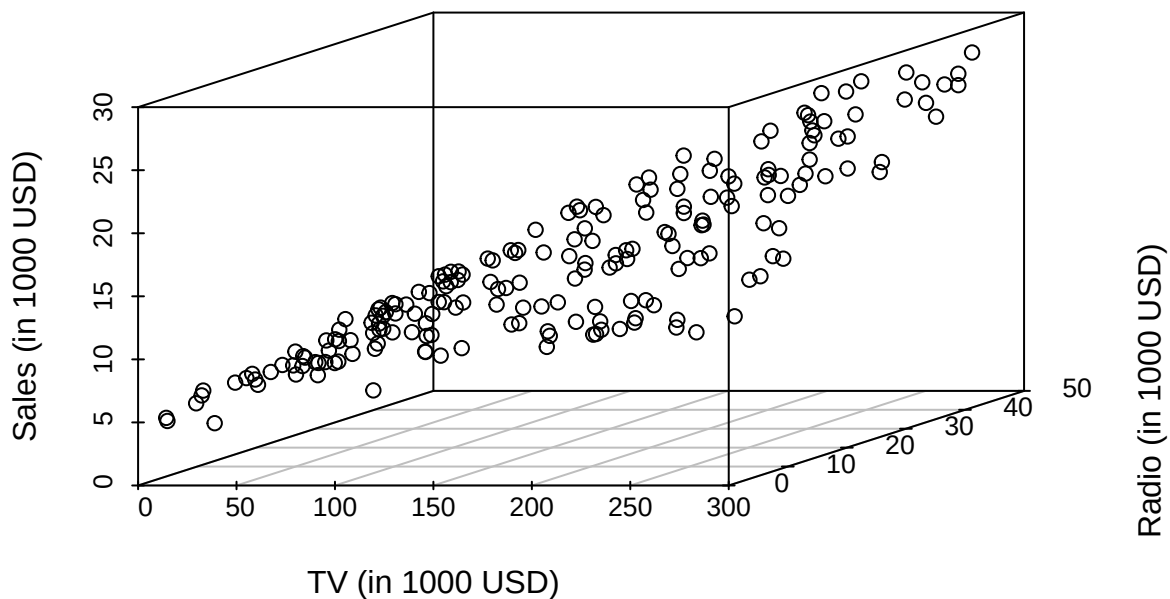
## Korrelation (Radio vs. Sales): 0.5762
```

```
# Korrelationsmatrix
cor_matrix <- cor(data[, c("TV", "radio", "newspaper", "sales")])
print(round(cor_matrix, 4))
```

```
##           TV  radio newspaper  sales
## TV       1.0000 0.0548   0.0566 0.7822
## radio    0.0548 1.0000   0.3541 0.5762
## newspaper 0.0566 0.3541   1.0000 0.2283
## sales    0.7822 0.5762   0.2283 1.0000
```

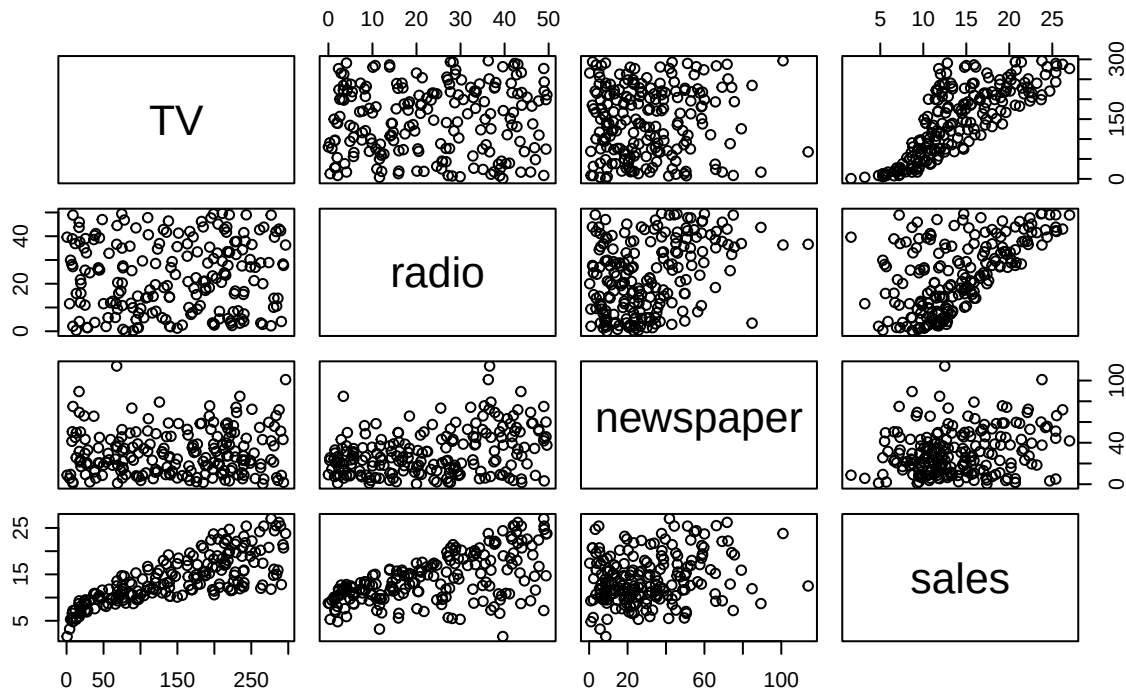
```
# 3D-Visualisierung (TV, Radio, Sales)
scatterplot3d(x = TV, y = radio, z = sales, scale.y = 0.9, angle = 30,
  main = "3D Scatterplot: TV, Radio und Sales",
  xlab = "TV (in 1000 USD)", ylab = "Radio (in 1000 USD)", zlab = "Sales (in
  ↪ 1000 USD)")
```

3D Scatterplot: TV, Radio und Sales



```
# Paarweise Streudiagramme
pairs(data[, c("TV", "radio", "newspaper", "sales")],
  main = "Paarweise Streudiagramme")
```

Paarweise Streudiagramme



```
# Dynamisches 3D-Scatterplot
# Hinweis: Diese Funktion öffnet ein interaktives Fenster
scatter3d(x = TV, z = radio, y = sales, surface = FALSE)
```

Statistische Erklärung:

- Die **Korrelation** ist ein Mass für den linearen Zusammenhang zwischen zwei Variablen und liegt zwischen -1 und +1.
- Eine **Korrelationsmatrix** zeigt die paarweisen Korrelationen zwischen mehreren Variablen.
- **3D-Streudiagramme** ermöglichen die Visualisierung des Zusammenhangs zwischen drei Variablen.
- **Paarweise Streudiagramme** zeigen die Beziehungen zwischen allen Paaren von Variablen in einem Datensatz.
- Bei der multiplen Regression ist es wichtig, die Beziehungen zwischen den Prädiktoren zu verstehen, da Multikollinearität (starke Korrelationen zwischen Prädiktoren) die Interpretation der Koeffizienten erschweren kann.

R-Erklärung:

- Die Funktion `cor()` berechnet die Pearson-Korrelation zwischen zwei Vektoren oder eine Korrelationsmatrix für einen Dataframe.
- Die Funktion `scatterplot3d()` aus dem gleichnamigen Paket erstellt ein 3D-Streudiagramm.
- Die Funktion `scatter3d()` aus dem Paket `car` erstellt ein interaktives 3D-Streudiagramm.
- Die Funktion `pairs()` erstellt eine Matrix von paarweisen Streudiagrammen.

4 Multiple lineare Regression (TV und Radio)

Nun führen wir eine multiple Regression mit zwei Prädiktoren durch: TV und Radio.

```
# Modell mit zwei Prädiktoren
```

```
md2 <- lm(sales ~ TV + radio)
```

```
summary(md2)
```

```
##
## Call:
## lm(formula = sales ~ TV + radio)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -8.7977 -0.8752  0.2422  1.1708  2.8328
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  2.92110    0.29449   9.919  <2e-16 ***
## TV           0.04575    0.00139  32.909  <2e-16 ***
## radio        0.18799    0.00804  23.382  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.681 on 197 degrees of freedom
## Multiple R-squared:  0.8972, Adjusted R-squared:  0.8962
## F-statistic: 859.6 on 2 and 197 DF,  p-value: < 2.2e-16
```

```
# Regressionsgleichung
```

```
B <- coef(md2)
```

```
cat("Regressionsgleichung:\n")
```

```
## Regressionsgleichung:
```

```
cat("sales ", round(B[1], 3), "+", round(B[2], 3), "* TV +", round(B[3], 3), "*
  ↪ radio\n\n")
```

```
## sales  2.921 + 0.046 * TV + 0.188 * radio
```

```
# Vergleich mit einfacher Regression (nur TV)
```

```
md1 <- lm(sales ~ TV)
```

```
summary(md1)
```

```
##
## Call:
## lm(formula = sales ~ TV)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -8.3860 -1.9545 -0.1913  2.0671  7.2124
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  7.032594    0.457843  15.36  <2e-16 ***
## TV           0.047537    0.002691  17.67  <2e-16 ***
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.259 on 198 degrees of freedom
## Multiple R-squared:  0.6119, Adjusted R-squared:  0.6099
## F-statistic: 312.1 on 1 and 198 DF,  p-value: < 2.2e-16
```

```
# Vergleich der Koeffizienten
```

```
cat("Koeffizient für TV (einfache Regression):", round(coef(md1)[2], 4), "\n")
```

```
## Koeffizient für TV (einfache Regression): 0.0475
```

```
cat("Koeffizient für TV (multiple Regression):", round(coef(md2)[2], 4), "\n\n")
```

```
## Koeffizient für TV (multiple Regression): 0.0458
```

```
# Interpretation der Koeffizienten
```

```
cat("Interpretation der Koeffizienten (multiple Regression):\n")
```

```
## Interpretation der Koeffizienten (multiple Regression):
```

```
cat("- Achsenabschnitt:", round(B[1], 3),
     "\n→ erwartete sales bei TV = 0 und radio = 0 (in Tausend USD)\n")
```

```
## - Achsenabschnitt: 2.921 → erwartete sales bei TV = 0 und radio = 0 (in Tausend USD)
```

```
cat("- TV-Koeffizient:", round(B[2], 4),
     "\n→ erwarteter Anstieg von sales bei +1 Einheit TV, wenn radio konstant bleibt\n")
```

```
## - TV-Koeffizient: 0.0458 → erwarteter Anstieg von sales bei +1 Einheit TV, wenn radio konstant bleibt
```

```
cat("- Radio-Koeffizient:", round(B[3], 4),
     "\n→ erwarteter Anstieg von sales bei +1 Einheit radio, wenn TV konstant bleibt\n")
```

```
## - Radio-Koeffizient: 0.188 → erwarteter Anstieg von sales bei +1 Einheit radio, wenn TV konstant bleibt
```

Statistische Erklärung:

- Die **multiple lineare Regression** modelliert den Zusammenhang zwischen einer abhängigen Variable und mehreren unabhängigen Variablen: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_k x_k + \epsilon$
- Die **Regressionskoeffizienten** ($\beta_1, \beta_2, \dots$) geben den Einfluss jeder unabhängigen Variable auf die abhängige Variable an, wenn alle anderen Variablen konstant gehalten werden.
- Der **Achsenabschnitt** (β_0) ist der erwartete Wert der abhängigen Variable, wenn alle unabhängigen Variablen Null sind.
- Die **Interpretation der Koeffizienten** in der multiplen Regression unterscheidet sich von der einfachen Regression:
 - In der einfachen Regression gibt der Koeffizient den Gesamteffekt der Variable an.
 - In der multiplen Regression gibt der Koeffizient den partiellen Effekt der Variable an, wenn alle anderen Variablen konstant gehalten werden.
- Die Koeffizienten können sich zwischen einfacher und multipler Regression unterscheiden, wenn die Prädiktoren korreliert sind.

R-Erklärung:

- Die Funktion `lm()` (linear model) passt ein lineares Regressionsmodell an die Daten an.

- Die Formel `sales ~ TV + radio` in `lm()` gibt an, dass `sales` die abhängige Variable und `TV` und `radio` die unabhängigen Variablen sind.
- Die Funktion `summary()` liefert eine detaillierte Zusammenfassung des Regressionsmodells, einschliesslich Koeffizienten, Standardfehler, t-Werte, p-Werte und R^2 .
- Die Funktion `coef()` extrahiert die Koeffizienten aus einem Regressionsmodell.

5 Vorhersage mit dem Modell

Mit dem Regressionsmodell können wir Vorhersagen für neue Werte der unabhängigen Variablen treffen.

```
# Vorhersage mit Fehlerbereich
RSE <- summary(md2)$sigma
tv_new <- 230.1
radio_new <- 37.8
predicted <- predict(md2, newdata = data.frame(TV = tv_new, radio = radio_new))
pred_interval <- predicted + c(-1, 1) * 2 * RSE
cat("Prognose für TV =", tv_new, "und Radio =", radio_new, ":", round(predicted, 2),
    ↵ "\n")
```

```
## Prognose für TV = 230.1 und Radio = 37.8 : 20.56
```

```
cat("95%-Prognoseintervall:", round(pred_interval[1], 2), "bis", round(pred_interval[2],
    ↵ 2), "\n\n")
```

```
## 95%-Prognoseintervall: 17.19 bis 23.92
```

```
# Exaktes Prognoseintervall
pred_interval_exact <- predict(md2, newdata = data.frame(TV = tv_new, radio = radio_new),
    ↵
                                interval = "prediction", level = 0.95)
cat("Exaktes 95%-Prognoseintervall:",
    round(pred_interval_exact[2], 2), "bis",
    round(pred_interval_exact[3], 2), "\n")
```

```
## Exaktes 95%-Prognoseintervall: 17.22 bis 23.89
```

```
# Konfidenzintervall für den Erwartungswert
conf_interval <- predict(md2, newdata = data.frame(TV = tv_new, radio = radio_new),
    ↵
                                interval = "confidence", level = 0.95)
cat("95%-Konfidenzintervall für den Erwartungswert:",
    round(conf_interval[2], 2), "bis",
    round(conf_interval[3], 2), "\n")
```

```
## 95%-Konfidenzintervall für den Erwartungswert: 20.16 bis 20.95
```

Statistische Erklärung:

- Die **Vorhersage** (Prognose) ist der vorhergesagte Wert der abhängigen Variable für bestimmte Werte der unabhängigen Variablen: $\hat{y} = + x + x + \dots + x$
- Der **Residual Standard Error (RSE)** ist ein Mass für die typische Grösse der Residuen und hat die gleiche Einheit wie die abhängige Variable.
- Das **Prognoseintervall** gibt den Bereich an, in dem ein einzelner neuer Wert mit einer bestimmten Wahrscheinlichkeit liegen wird.

- Das **Konfidenzintervall für den Erwartungswert** gibt den Bereich an, in dem der wahre Erwartungswert mit einer bestimmten Wahrscheinlichkeit liegt.
- Prognoseintervalle sind breiter als Konfidenzintervalle für den Erwartungswert, da sie zusätzlich die Variation der einzelnen Beobachtungen um den Erwartungswert berücksichtigen.

R-Erklärung:

- Die Funktion `predict()` berechnet vorhergesagte Werte für ein Regressionsmodell.
- Der Parameter `newdata` in `predict()` gibt einen Dataframe mit den Werten der Prädiktoren an, für die Vorhersagen gemacht werden sollen.
- Der Parameter `interval` in `predict()` gibt an, ob Konfidenz- oder Prognoseintervalle berechnet werden sollen.
- Der Parameter `level` in `predict()` gibt das Konfidenzniveau an.
- Der Ausdruck `summary(md2)$sigma` extrahiert den RSE aus dem Regressionsobjekt.

6 Regressionsfläche (Plane of Best Fit)

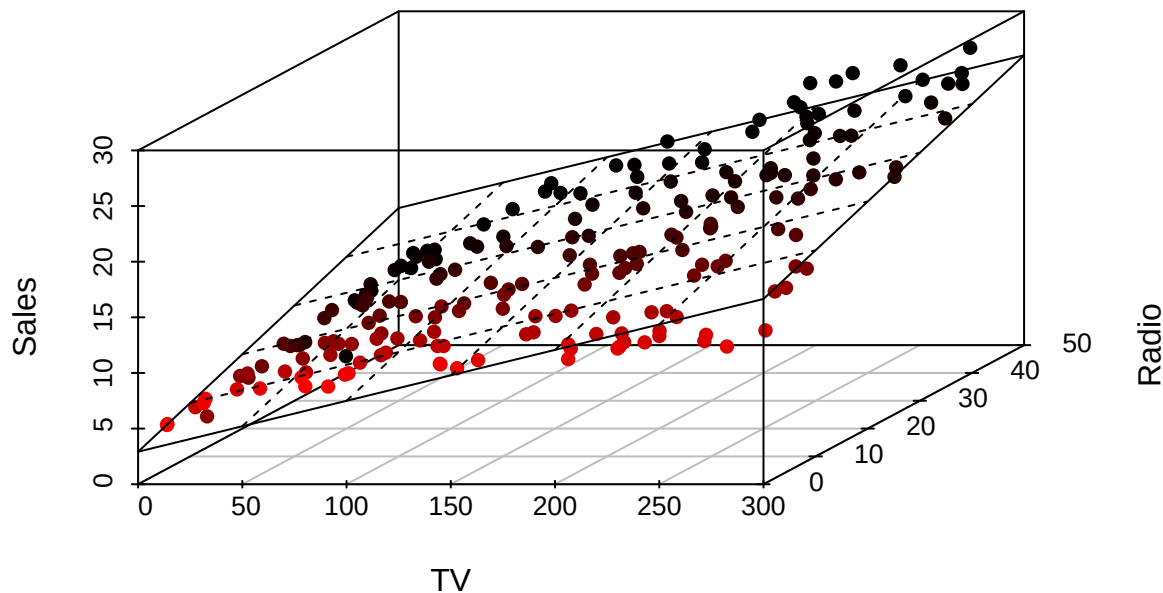
Bei der multiplen Regression mit zwei Prädiktoren können wir die Regressionsfläche in einem 3D-Raum visualisieren.

```
# Visualisierung der Regressionsfläche  
# Hinweis: Diese Funktion öffnet ein interaktives Fenster  
scatter3d(x = TV, z = radio, y = sales, fit = "linear", residuals = TRUE, grid = TRUE)
```

```
# Statische Visualisierung der Regressionsfläche  
s3d <- scatterplot3d(TV, radio, sales,  
                    main = "Regressionsfläche (Plane of Best Fit)",  
                    xlab = "TV", ylab = "Radio", zlab = "Sales",  
                    pch = 16, highlight.3d = TRUE, angle = 45)
```

```
# Regressionsfläche hinzufügen  
my.lm <- lm(sales ~ TV + radio)  
s3d$plane3d(my.lm, lty.box = "solid")
```


Regressionsfläche (Plane of Best Fit)



Statistische Erklärung:

- Bei der **multiplen Regression mit zwei Prädiktoren** wird der Zusammenhang durch eine Ebene im dreidimensionalen Raum dargestellt: $z = \beta_0 + \beta_1 x + \beta_2 y$
- Diese Ebene wird als **Regressionsfläche** (Plane of Best Fit) bezeichnet.
- Die Regressionsfläche wird so gewählt, dass die Summe der quadrierten vertikalen Abstände der Datenpunkte zur Fläche minimiert wird.
- Die **Residuen** sind die vertikalen Abstände der Datenpunkte zur Regressionsfläche.
- Die Visualisierung der Regressionsfläche hilft, den gemeinsamen Einfluss der beiden Prädiktoren auf die abhängige Variable zu verstehen.

R-Erklärung:

- Die Funktion `scatter3d()` aus dem Paket `car` erstellt ein interaktives 3D-Streudiagramm mit einer Regressionsfläche.
- Der Parameter `fit = "linear"` in `scatter3d()` gibt an, dass eine lineare Regressionsfläche angepasst werden soll.
- Der Parameter `residuals = TRUE` in `scatter3d()` zeigt die Residuen als vertikale Linien an.
- Die Funktion `scatterplot3d()` erstellt ein statisches 3D-Streudiagramm.
- Die Methode `plane3d()` fügt eine Regressionsfläche zu einem `scatterplot3d`-Objekt hinzu.

7 Modellgüte und Erklärungskraft

Wir untersuchen nun, wie gut das Modell die Variation in den Daten erklärt.

```
# Berechnung von TSS, RSS und R²
TSS <- sum((sales - mean(sales))^2)
RSS <- sum((sales - predict(md2))^2)
r_squared <- 1 - (RSS / TSS)

cat("Bestimmtheitsmass (R²):", round(r_squared, 4), "\n")
```

```
## Bestimmtheitsmass ( $R^2$ ): 0.8972

cat("TV und Radio erklären", round(r_squared * 100, 2), "% der Variation in Sales.\n\n")

## TV und Radio erklären 89.72 % der Variation in Sales.

# Vergleich mit  $R^2$  aus summary()
r_squared_summary <- summary(md2)$r.squared
cat(" $R^2$  aus summary():", round(r_squared_summary, 4), "\n")

##  $R^2$  aus summary(): 0.8972

# Vergleich mit einfacher Regression (nur TV)
r_squared_tv <- summary(md1)$r.squared
cat(" $R^2$  (nur TV):", round(r_squared_tv, 4), "\n")

##  $R^2$  (nur TV): 0.6119

cat(" $R^2$  (TV + Radio):", round(r_squared_summary, 4), "\n")

##  $R^2$  (TV + Radio): 0.8972

cat("Zunahme von  $R^2$  durch Hinzufügen von Radio:",
    round((r_squared_summary - r_squared_tv) * 100, 2), "Prozentpunkte\n")

## Zunahme von  $R^2$  durch Hinzufügen von Radio: 28.53 Prozentpunkte
```

Statistische Erklärung:

- Das **Bestimmtheitsmass R^2** gibt den Anteil der Variation in der abhängigen Variable an, der durch das Regressionsmodell erklärt wird.
- R^2 liegt zwischen 0 und 1:
 - $R^2 = 0$: Das Modell erklärt keine Variation in den Daten.
 - $R^2 = 1$: Das Modell erklärt die gesamte Variation in den Daten.
- R^2 kann auf verschiedene Weisen berechnet werden:
 - $R^2 = 1 - (\text{RSS} / \text{TSS})$
 - $R^2 = \text{ESS} / \text{TSS}$
- Ein höheres R^2 bedeutet eine bessere Anpassung des Modells an die Daten.
- Durch Hinzufügen weiterer Prädiktoren kann R^2 nur zunehmen oder gleich bleiben, nie abnehmen.
- Dies kann zu einer Überanpassung (Overfitting) führen, wenn zu viele Prädiktoren hinzugefügt werden.

R-Erklärung:

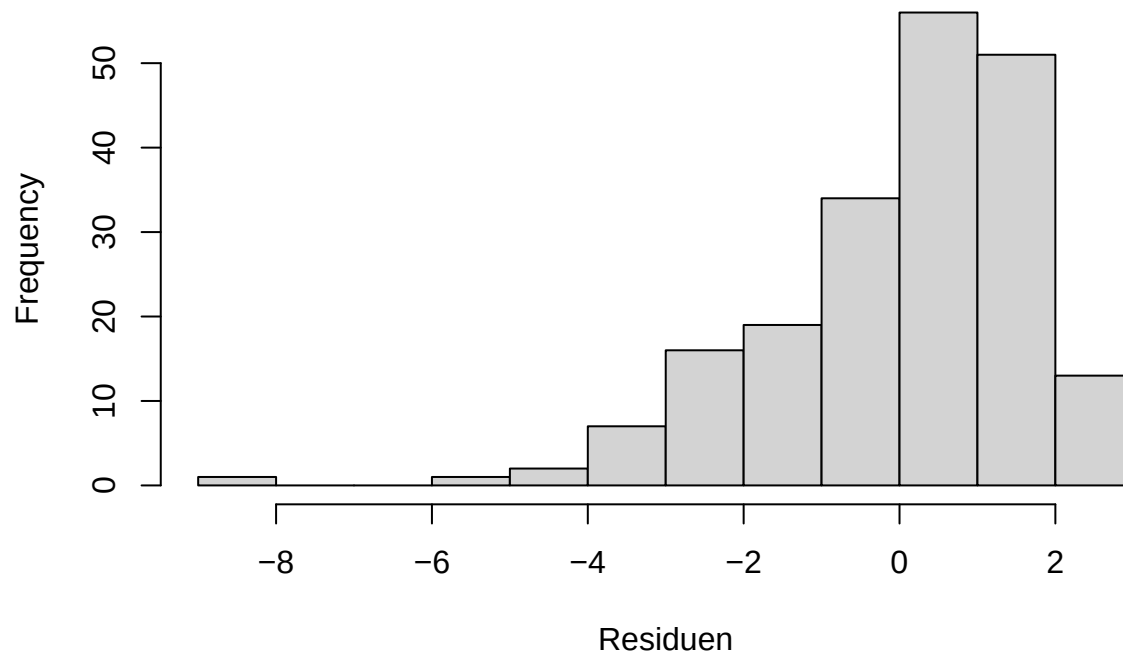
- Die Funktion `predict()` ohne weitere Parameter berechnet die vorhergesagten Werte für die Beobachtungen im Trainingsdatensatz.
- Der Ausdruck `summary(md2)$r.squared` extrahiert den R^2 -Wert aus dem Regressionsobjekt.
- Die Berechnung von R^2 erfolgt direkt nach der Formel $R^2 = 1 - (\text{RSS} / \text{TSS})$.

8 Residuenanalyse

Die Analyse der Residuen ist ein wichtiger Schritt, um die Annahmen der linearen Regression zu überprüfen.

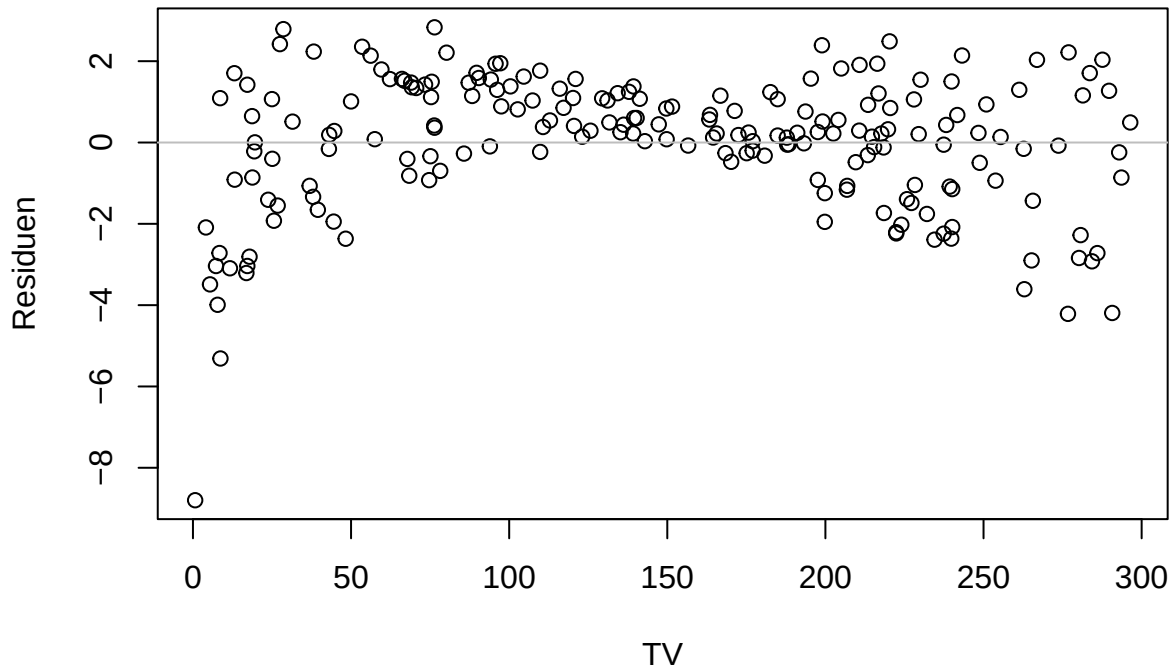
```
# Residuenplots  
hist(resid(md2), main = "Histogramm der Residuen", xlab = "Residuen")
```

Histogramm der Residuen



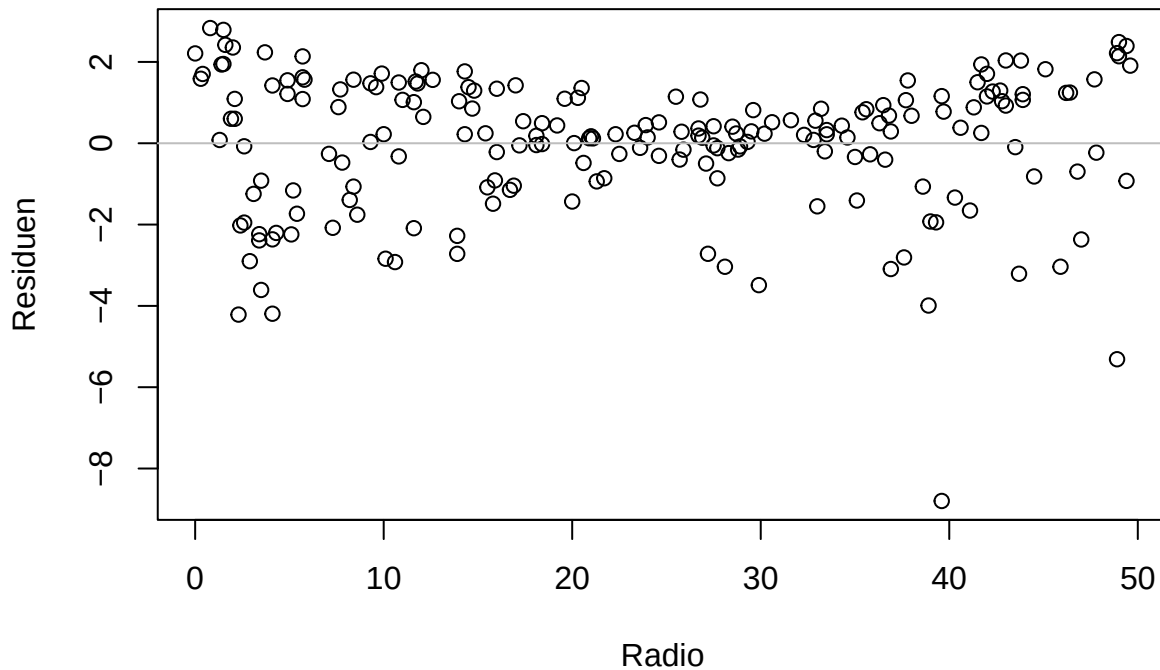
```
plot(resid(md2) ~ TV, main = "Residuen vs. TV", xlab = "TV", ylab = "Residuen")  
abline(h = 0, col = "gray")
```

Residuen vs. TV



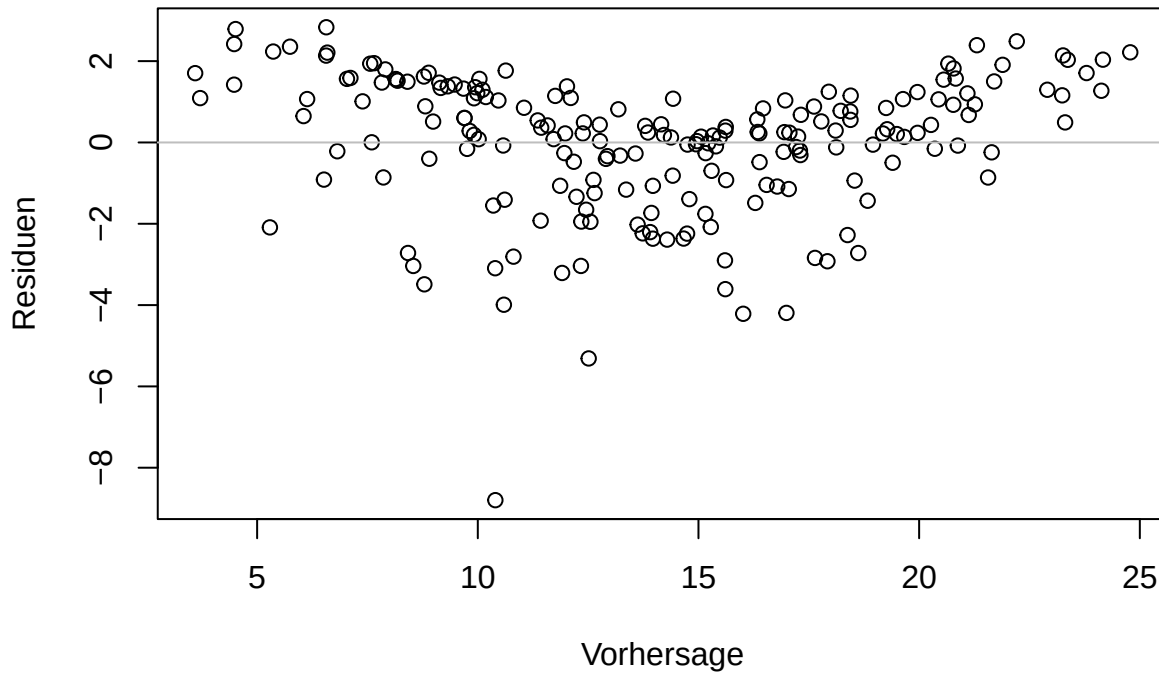
```
plot(resid(md2) ~ radio, main = "Residuen vs. Radio", xlab = "Radio", ylab = "Residuen")  
abline(h = 0, col = "gray")
```

Residuen vs. Radio



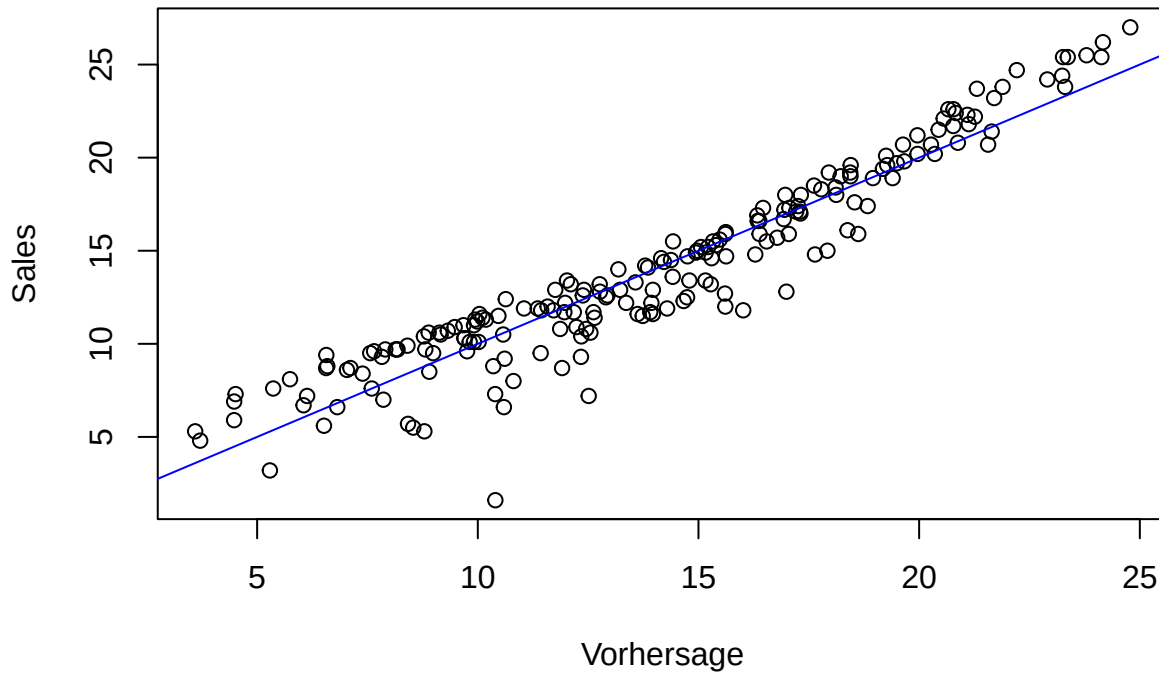
```
# Residuen vs. Vorhergesagte Werte  
plot(resid(md2) ~ predict(md2), main = "Residuen vs. Vorhergesagte Werte", xlab =  
  ↪ "Vorhersage", ylab = "Residuen")  
abline(h = 0, col = "gray")
```

Residuen vs. Vorhergesagte Werte



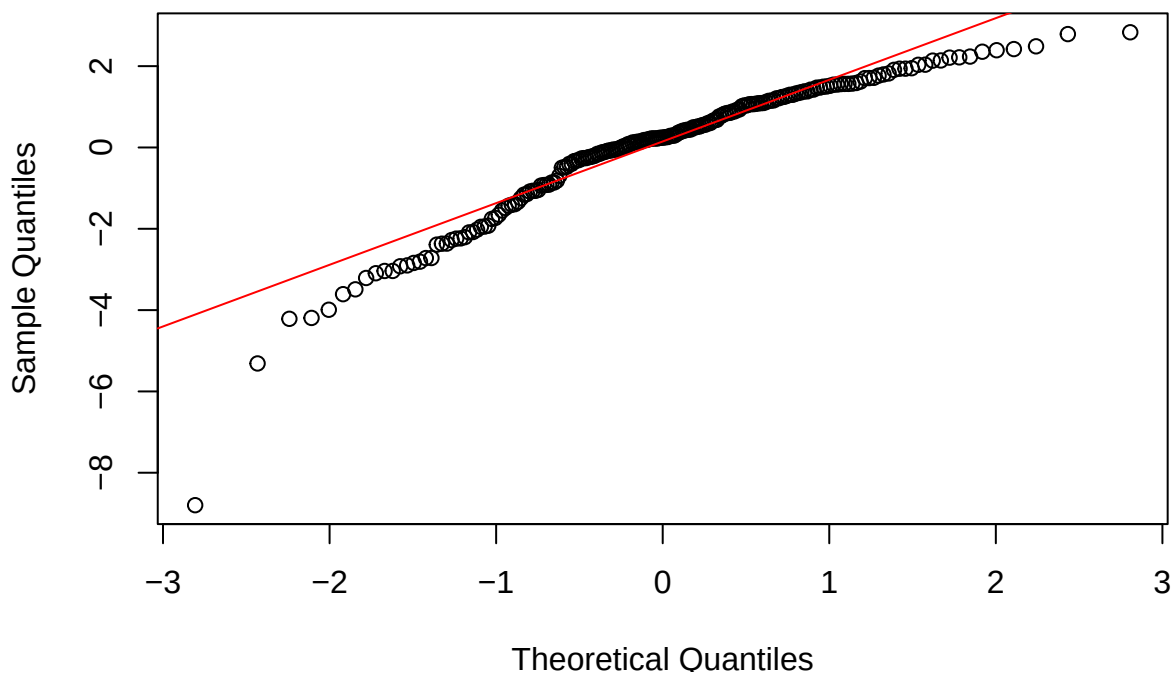
```
# Kalibrierungsplot  
plot(sales ~ predict(md2), main = "Kalibrierungsplot: sales vs. Vorhersage", xlab =  
  ↪ "Vorhersage", ylab = "Sales")  
abline(0, 1, col = "blue")
```

Kalibrierungsplot: sales vs. Vorhersage



```
# QQ-Plot der Residuen  
qqnorm(resid(md2), main = "Q-Q Plot der Residuen")  
qqline(resid(md2), col = "red")
```

Q-Q Plot der Residuen



Statistische Erklärung:

- Die **Residuenanalyse** dient der Überprüfung der Annahmen der linearen Regression:
 - Normalität: Die Residuen sollten normalverteilt sein.
 - Homoskedastizität: Die Varianz der Residuen sollte konstant sein.
 - Unabhängigkeit: Die Residuen sollten unabhängig voneinander sein.
 - Linearität: Der Zusammenhang zwischen den Prädiktoren und der abhängigen Variable sollte linear sein.
- **Residuenplots** zeigen die Residuen in Abhängigkeit von den Prädiktoren oder den vorhergesagten Werten.
- Ein **Histogramm der Residuen** gibt Aufschluss über die Verteilung der Residuen.
- Ein **Q-Q-Plot** vergleicht die Verteilung der Residuen mit der Normalverteilung.
- Ein **Kalibrierungsplot** zeigt die beobachteten Werte in Abhängigkeit von den vorhergesagten Werten. Im Idealfall liegen die Punkte auf der Diagonalen.

R-Erklärung:

- Die Funktion `resid()` extrahiert die Residuen aus einem Regressionsmodell.
- Die Funktion `hist()` erstellt ein Histogramm.
- Die Funktion `plot()` mit einer Formel erstellt ein Streudiagramm.
- Die Funktion `abline()` mit dem Parameter `h` fügt eine horizontale Linie zum Plot hinzu.
- Die Funktion `qqnorm()` erstellt einen Q-Q-Plot, der die Verteilung der Residuen mit der Normalverteilung vergleicht.
- Die Funktion `qqline()` fügt eine Referenzlinie zum Q-Q-Plot hinzu.

9 Erweiterung: Multiple Regression mit drei Prädiktoren

Wir erweitern nun das Modell um einen dritten Prädiktor: Newspaper.

```
# Modell mit TV, Radio und Newspaper
md3 <- lm(sales ~ TV + radio + newspaper)
summary(md3)
```

```
##
## Call:
## lm(formula = sales ~ TV + radio + newspaper)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -8.8277 -0.8908  0.2418  1.1893  2.8292
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  2.938889   0.311908   9.422  <2e-16 ***
## TV           0.045765   0.001395  32.809  <2e-16 ***
## radio        0.188530   0.008611  21.893  <2e-16 ***
## newspaper   -0.001037   0.005871  -0.177    0.86
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.686 on 196 degrees of freedom
## Multiple R-squared:  0.8972, Adjusted R-squared:  0.8956
## F-statistic: 570.3 on 3 and 196 DF,  p-value: < 2.2e-16
```

```
# Vergleich von R2 und RSE
cat("Modellvergleich:\n")
```

```
## Modellvergleich:
```

```
cat("R2 (TV + Radio):", round(summary(md2)$r.squared, 4), "\n")
```

```
## R2 (TV + Radio): 0.8972
```

```
cat("R2 (TV + Radio + Newspaper):", round(summary(md3)$r.squared, 4), "\n")
```

```
## R2 (TV + Radio + Newspaper): 0.8972
```

```
cat("RSE (TV + Radio):", round(summary(md2)$sigma, 4), "\n")
```

```
## RSE (TV + Radio): 1.6814
```

```
cat("RSE (TV + Radio + Newspaper):", round(summary(md3)$sigma, 4), "\n\n")
```

```
## RSE (TV + Radio + Newspaper): 1.6855
```

```
# Adjustiertes R2
```

```
cat("Adjustiertes R2 (TV + Radio):", round(summary(md2)$adj.r.squared, 4), "\n")
```

```
## Adjustiertes R2 (TV + Radio): 0.8962
```

```
cat("Adjustiertes R2 (TV + Radio + Newspaper):", round(summary(md3)$adj.r.squared, 4),
    "\n")
```

```
## Adjustiertes R2 (TV + Radio + Newspaper): 0.8956
```

Statistische Erklärung:

- Bei der **multiplen Regression mit drei Prädiktoren** wird der Zusammenhang durch eine Hyper-ebene im vierdimensionalen Raum dargestellt: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3$
- Das **adjustierte R²** berücksichtigt die Anzahl der Prädiktoren im Modell und bestraft Modelle mit zu vielen Prädiktoren:
 - $\text{Adj. } R^2 = 1 - [(1 - R^2) * (n - 1) / (n - p - 1)]$
 - n ist die Anzahl der Beobachtungen
 - p ist die Anzahl der Prädiktoren
- Das adjustierte R² kann abnehmen, wenn ein hinzugefügter Prädiktor wenig zur Erklärungskraft des Modells beiträgt.
- Der **Residual Standard Error (RSE)** ist ein Mass für die typische Grösse der Residuen und sollte bei einem besseren Modell kleiner sein.
- In diesem Beispiel führt das Hinzufügen von Newspaper zu einer minimalen Verbesserung von R², aber das adjustierte R² nimmt sogar leicht ab, was darauf hindeutet, dass Newspaper keinen wesentlichen Beitrag zur Erklärungskraft des Modells leistet.

R-Erklärung:

- Die Formel `sales ~ TV + radio + newspaper` in `lm()` gibt an, dass `sales` die abhängige Variable und `TV`, `radio` und `newspaper` die unabhängigen Variablen sind.
- Der Ausdruck `summary(md2)$adj.r.squared` extrahiert das adjustierte R² aus dem Regressionsobjekt.

- Der Ausdruck `summary(md2)$sigma` extrahiert den RSE aus dem Regressionsobjekt.

10 Nichtlineare Modelle: Quadratische und polynomiale Regression

Nicht alle Zusammenhänge sind linear. Wir untersuchen nun nichtlineare Modelle, insbesondere quadratische und polynomiale Regression.

```
# Quadratische Regression
```

```
md.q2 <- lm(sales ~ TV + I(TV^2) + radio)
summary(md.q2)
```

```
##
## Call:
## lm(formula = sales ~ TV + I(TV^2) + radio)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -7.3860 -0.8822 -0.0498  0.9613  3.5725
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.288e+00  3.588e-01   3.588 0.000421 ***
## TV           7.844e-02  4.985e-03  15.736 < 2e-16 ***
## I(TV^2)      -1.136e-04  1.677e-05  -6.775 1.42e-10 ***
## radio        1.930e-01  7.293e-03  26.465 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.517 on 196 degrees of freedom
## Multiple R-squared:  0.9167, Adjusted R-squared:  0.9154
## F-statistic: 719 on 3 and 196 DF, p-value: < 2.2e-16
```

```
# Polynomiale Regression (Grad 3)
```

```
md.q3 <- lm(sales ~ poly(TV, 3) + radio)
summary(md.q3)
```

```
##
## Call:
## lm(formula = sales ~ poly(TV, 3) + radio)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -6.1892 -0.8451 -0.0614  0.7654  3.7264
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   9.464052   0.189302  49.994 < 2e-16 ***
## poly(TV, 3)1  55.323451   1.429492  38.701 < 2e-16 ***
## poly(TV, 3)2 -10.394968   1.434792  -7.245 9.82e-12 ***
## poly(TV, 3)3   7.373480   1.432197   5.148 6.39e-07 ***
## radio          0.195944   0.006884  28.463 < 2e-16 ***
```

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.427 on 195 degrees of freedom
## Multiple R-squared:  0.9267, Adjusted R-squared:  0.9252
## F-statistic:    616 on 4 and 195 DF,  p-value: < 2.2e-16

# Modellvergleich: Adjustiertes R2
cat("Modellvergleich (nichtlinear):\n")

## Modellvergleich (nichtlinear):

cat("R2 (linear):", round(summary(md2)$adj.r.squared, 4), "\n")

## R2 (linear): 0.8962

cat("R2 (quadratisch):", round(summary(md.q2)$adj.r.squared, 4), "\n")

## R2 (quadratisch): 0.9154

cat("R2 (kubisch):", round(summary(md.q3)$adj.r.squared, 4), "\n\n")

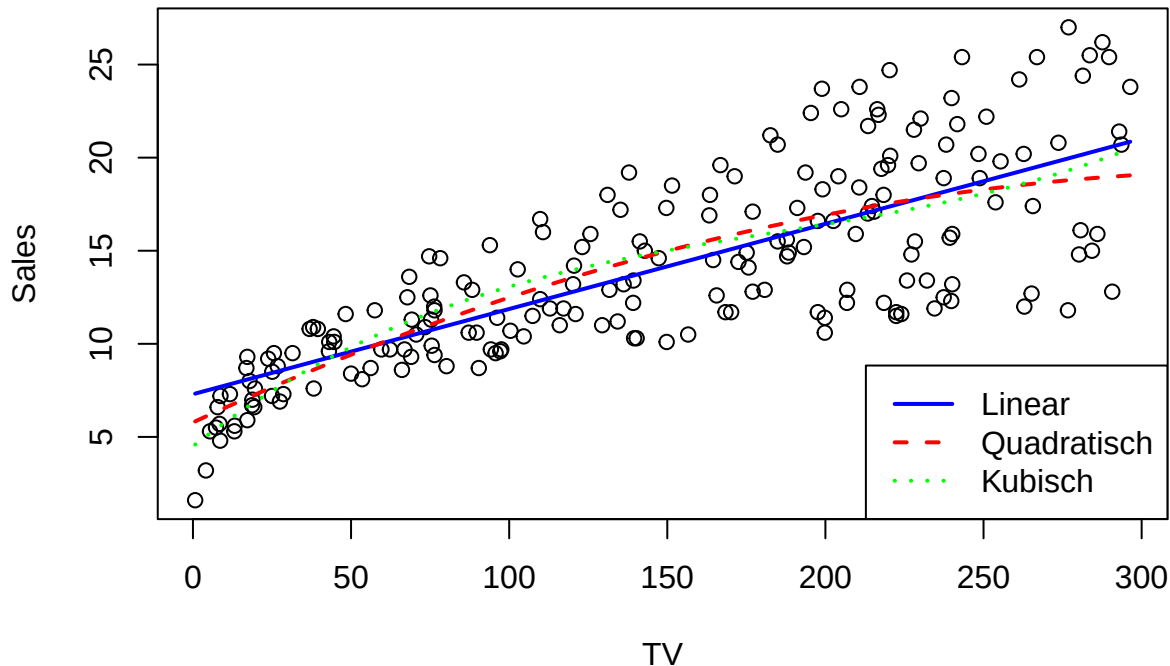
## R2 (kubisch): 0.9252

# Visualisierung der Modelle
# Erstelle ein Gitter von TV-Werten
tv_grid <- seq(min(TV), max(TV), length.out = 100)
radio_mean <- mean(radio)

# Vorhersagen für die verschiedenen Modelle
pred_linear <- predict(md2, newdata = data.frame(TV = tv_grid, radio = radio_mean))
pred_quad <- predict(md.q2, newdata = data.frame(TV = tv_grid, radio = radio_mean))
pred_poly3 <- predict(md.q3, newdata = data.frame(TV = tv_grid, radio = radio_mean))

# Plot
plot(TV, sales, main = "Vergleich der Modelle", xlab = "TV", ylab = "Sales")
lines(tv_grid, pred_linear, col = "blue", lwd = 2)
lines(tv_grid, pred_quad, col = "red", lwd = 2, lty = 2)
lines(tv_grid, pred_poly3, col = "green", lwd = 2, lty = 3)
legend("bottomright",
      legend = c("Linear", "Quadratisch", "Kubisch"),
      col = c("blue", "red", "green"),
      lty = c(1, 2, 3),
      lwd = 2)
```

Vergleich der Modelle



Statistische Erklärung:

- **Nichtlineare Regression** modelliert nichtlineare Zusammenhänge zwischen den Prädiktoren und der abhängigen Variable.
- **Quadratische Regression** fügt einen quadratischen Term hinzu: $y = \dots + x + x^2 + \dots$
- **Polynomiale Regression** verallgemeinert dies auf höhere Potenzen: $y = \dots + x + x^2 + x^3 + \dots + x^n + \dots$
- Die Funktion `I(TV^2)` in R fügt einen quadratischen Term hinzu, wobei die ursprüngliche Skala beibehalten wird.
- Die Funktion `poly(TV, 3)` fügt orthogonale Polynome bis zum Grad 3 hinzu, was numerisch stabiler ist.
- Nichtlineare Modelle können komplexere Zusammenhänge erfassen, aber sie sind auch anfälliger für Überanpassung (Overfitting).
- Die Wahl zwischen linearen und nichtlinearen Modellen sollte auf der Natur des Problems, der Visualisierung der Daten und Modellvergleichskriterien wie dem adjustierten R^2 basieren.

R-Erklärung:

- Die Funktion `I()` in einer Formel schützt den Ausdruck vor der Interpretation durch die Formelsprache.
- Die Funktion `poly()` erzeugt orthogonale Polynome, die numerisch stabiler sind als einfache Potenzen.
- Die Funktion `seq()` erzeugt eine Sequenz von Werten.
- Die Funktion `lines()` fügt Linien zu einem bestehenden Plot hinzu.
- Die Parameter `col`, `lwd` und `lty` in `lines()` geben die Farbe, Breite und den Typ der Linie an.

11 Modellvergleich: R^2 , Adjustiertes R^2 und RSE

Wir vergleichen nun systematisch die verschiedenen Modelle anhand von R^2 , adjustiertem R^2 und RSE.

```
# Modellvergleich:  $R^2$ , Adjustiertes  $R^2$  und RSE
models <- list()
```

```

"TV" = md1,
"TV + Radio" = md2,
"TV + Radio + Newspaper" = md3,
"TV + TV2 + Radio" = md.q2,
"TV (Poly3) + Radio" = md.q3
)

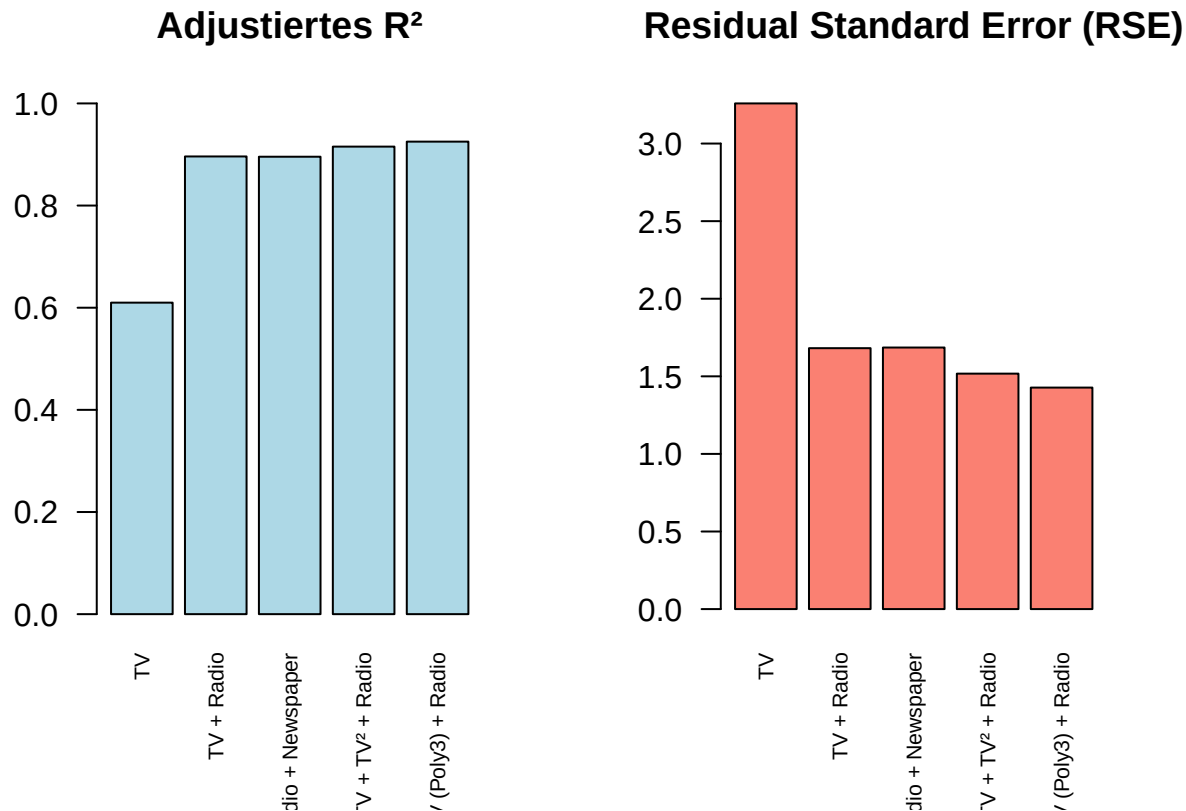
# Extrahiere R2, Adj. R2 und RSE für jedes Modell
model_stats <- data.frame(
  Model = names(models),
  R2 = sapply(models, function(m) summary(m)$r.squared),
  Adj_R2 = sapply(models, function(m) summary(m)$adj.r.squared),
  RSE = sapply(models, function(m) summary(m)$sigma)
)

# Ausgabe der Modelstatistiken
# Round only the numeric columns
model_stats_rounded <- model_stats
model_stats_rounded[, 2:4] <- round(model_stats[, 2:4], 4)
print(model_stats_rounded)

##                               Model      R2 Adj_R2    RSE
## TV                               TV 0.6119 0.6099 3.2587
## TV + Radio                       TV + Radio 0.8972 0.8962 1.6814
## TV + Radio + Newspaper TV + Radio + Newspaper 0.8972 0.8956 1.6855
## TV + TV2 + Radio               TV + TV2 + Radio 0.9167 0.9154 1.5173
## TV (Poly3) + Radio              TV (Poly3) + Radio 0.9267 0.9252 1.4273

# Visualisierung des Modellvergleichs
par(mfrow = c(1, 2))
barplot(model_stats$Adj_R2, names.arg = model_stats$Model,
  main = "Adjustiertes R2", las = 2, cex.names = 0.7,
  col = "lightblue", ylim = c(0, 1))
barplot(model_stats$RSE, names.arg = model_stats$Model,
  main = "Residual Standard Error (RSE)", las = 2, cex.names = 0.7,
  col = "salmon")

```



```
par(mfrow = c(1, 1))
```

Statistische Erklärung:

- Der **Modellvergleich** ist ein wichtiger Schritt bei der Auswahl des besten Modells für die Daten.
- R^2 gibt den Anteil der erklärten Variation an, aber es steigt immer, wenn Prädiktoren hinzugefügt werden, auch wenn diese nicht informativ sind.
- Das **adjustierte R^2** berücksichtigt die Anzahl der Prädiktoren und bestraft Modelle mit zu vielen Prädiktoren.
- Der **Residual Standard Error (RSE)** gibt die typische Grösse der Residuen an und sollte bei einem besseren Modell kleiner sein.
- Bei der Modellauswahl sollte man einen Kompromiss zwischen Anpassungsgüte und Modellkomplexität finden:
 - Ein zu einfaches Modell kann wichtige Zusammenhänge übersehen (Underfitting).
 - Ein zu komplexes Modell kann Rauschen modellieren und schlecht generalisieren (Overfitting).
- In diesem Beispiel scheint das Modell "TV + TV² + Radio" den besten Kompromiss zu bieten, da es das höchste adjustierte R^2 und einen niedrigen RSE hat.

R-Erklärung:

- Die Funktion `list()` erstellt eine Liste von Objekten.
- Die Funktion `sapply()` wendet eine Funktion auf jedes Element einer Liste an und gibt einen Vektor zurück.
- Die Funktion `par(mfrow = c(1, 2))` teilt das Grafikfenster in 1 Zeile und 2 Spalten.
- Die Funktion `barplot()` erstellt ein Balkendiagramm.
- Der Parameter `las = 2` in `barplot()` dreht die Achsenbeschriftungen.
- Der Parameter `cex.names = 0.7` in `barplot()` verkleinert die Schriftgrösse der Namen.

12 Zusammenfassung

In dieser Vorlesung haben wir die multiple lineare Regression kennengelernt, eine Erweiterung der einfachen linearen Regression auf mehrere Prädiktoren. Die wichtigsten Konzepte waren:

1. **Multiple Regression mit mehreren Prädiktoren:** Die multiple Regression modelliert den Zusammenhang zwischen einer abhängigen Variable und mehreren unabhängigen Variablen. Die Koeffizienten geben den partiellen Effekt jeder Variable an, wenn alle anderen Variablen konstant gehalten werden.
2. **Residuenanalyse und Modellgüte:** Die Analyse der Residuen hilft, die Annahmen der linearen Regression zu überprüfen. Die Modellgüte wird durch Masse wie R^2 , adjustiertes R^2 und RSE quantifiziert.
3. **Prognose und Fehlerbereich:** Mit dem Regressionsmodell können Vorhersagen für neue Werte der unabhängigen Variablen getroffen werden. Prognose- und Konfidenzintervalle quantifizieren die Unsicherheit dieser Vorhersagen.
4. **Nichtlineare und polynomiale Regression:** Nicht alle Zusammenhänge sind linear. Quadratische und polynomiale Regression können komplexere Beziehungen modellieren.
5. **Modellvergleich:** Die Wahl zwischen verschiedenen Modellen sollte auf Kriterien wie dem adjustierten R^2 und dem RSE basieren, wobei ein Kompromiss zwischen Anpassungsgüte und Modellkomplexität gefunden werden sollte.

Die multiple Regression ist ein mächtiges Werkzeug für die Datenanalyse, hat aber auch Einschränkungen. Sie setzt lineare Beziehungen voraus (es sei denn, nichtlineare Terme werden explizit hinzugefügt), kann durch Multikollinearität beeinträchtigt werden und ist anfällig für Ausreisser. In solchen Fällen können robustere oder flexiblere Methoden angemessener sein.